CSCI 6461 | TOPIC 3

The Hardware/Software Interface & AArch64 Programmer's Model

Architectural state, registers, memory organization, and foundational instruction mechanics

Dr. J. Burns
Computer Systems Architecture — AArch64 Reference Platform

Topic 3: Module Topics

1. **Architectural State & The Hardware Contract**
   - Concrete definition of architectural state vs. microarchitectural structures.
   - The AArch64 programmer's model and processor execution modes.
2. **AArch64 Register File & Naming Conventions**
   - The 31 general-purpose registers: 64-bit (`x0`–`x30`) vs. 32-bit (`w0`–`w30`).
   - Special-purpose registers: Zero register (`xzr`/`wzr`), stack pointer (`sp`), program counter (`pc`).
   - Standard ABI register usage: arguments, return values, caller-saved, and callee-saved registers.
3. **Memory Organization & Endianness**
   - Byte-addressable 64-bit address space, memory alignment rules, and byte ordering.
4. **Core Instruction Classes & Execution Mechanics**
   - Data movement, arithmetic/logical operations, NZCV condition flags, and condition codes.

PART 1

Architectural State & Execution Model

What Is Architectural State?

- **Definition:** The software-visible processor state needed to describe a program's execution at a particular point.
- On a context switch or exception, saving and later restoring the required architectural state allows execution to resume with the same program-visible behavior.
- **Core Components in AArch64:**
  - General-Purpose Registers (`x0`–`x30`).
  - Program Counter (`pc`) and Stack Pointer (`sp`).
  - Processor state and condition flags (`pstate` / NZCV).
  - System control and configuration registers (e.g., TTBR0_EL1, SCTLR_EL1 for memory management).
  - Architecturally visible memory state associated with the executing program.

State Checkpointing

Operating systems rely on architectural state to suspend and resume execution while preserving program behavior.

Architectural vs. Microarchitectural State

Architectural State (Visible Contract)

- Defined strictly by the ISA specification.
- Software directly reads/writes this state.
- **Examples:** Registers `x0`–`x30`, `pc`, architectural memory, NZCV flags.

Microarchitectural State (Hidden)

- Internal hardware implementation details.
- Invisible to software correctness.
- **Examples:** Pipeline latches, L1/L2 caches, Branch Target Buffers, rename tables.

The Execution Boundary

Hardware may execute instructions out of order internally, but retirement must preserve the architectural behavior required by program order.

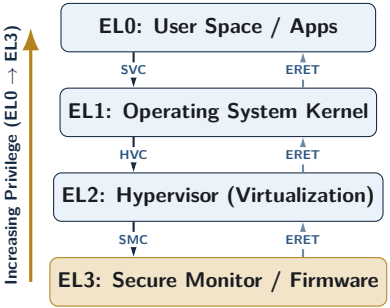
The AArch64 Execution State

- **AArch64:** The 64-bit execution state introduced in ARMv8-A.
- **64-Bit Address Representation:** General-purpose registers can hold 64-bit pointers, while implemented virtual-address sizes are smaller and configuration-dependent. A64 instructions are fixed 32-bit words aligned to 4-byte boundaries.
- **Distinct A64 Instruction Set:** A64 defines a separate encoding model from A32, removes broad per-instruction conditional execution, and keeps the Program Counter (pc) outside the general-purpose register file.

A64 Design Consequence

A64 is a distinct instruction set from A32, allowing ARM to simplify instruction encoding and remove several older architectural conventions while preserving compatibility through separate execution states on implementations that support them.

Privilege Levels (Exception Levels)



CSCI 6461 Scope

Analysis focuses on **EL0** execution mechanics and the **EL0** → **EL1** system call boundary. The diagram shows typical paths; exact exception routing is configuration-dependent.

- **EL0 (Application):** Unprivileged user space executing application logic with private virtual address spaces.
- **EL1 (OS Kernel):** Privileged kernel execution managing memory translation, drivers, and process scheduling
- **EL2 (Hypervisor):** Non-secure virtualization layer enforcing second-stage address translation across guest OS instances
- **EL3 (Secure Monitor):** Secure monitor / firmware execution level used for selected security-state transitions and platform firmware

PART 2

AArch64 Register Architecture & ABI

General-Purpose Registers: x0–x30

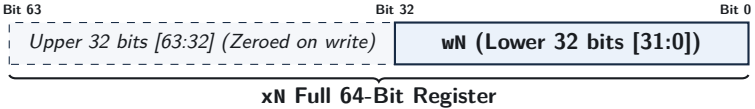
- AArch64 provides **31 general-purpose registers** (x0–x30), each exactly **64 bits wide** (8 bytes).
- **Highly Regular Design:** Most instructions allow general-purpose registers to be used uniformly as sources and destinations, with specific exceptions depending on the instruction encoding.
- **Regular Operand Model:** AArch64 avoids many legacy implicit-register conventions, although some instructions still assign special roles to particular registers.

Why Exactly 31 Registers?

Many A64 register fields are 5 bits wide ($2^5 = 32$ encodings). Encoding value 31 denotes either the zero register (xzr/wzr) or the stack pointer (sp/wsp) depending on the instruction class.

32-Bit Register Views: w0–w30

- Every 64-bit x register has a corresponding 32-bit view denoted as w0 through w30.
- wN represents the **lower 32 bits (bits 31:0)** of the physical register xN.
- Enables native 32-bit integer arithmetic without requiring separate hardware register files.



Zero-Extension Safety Rule

Writing to a 32-bit w register automatically zeroes bits 63:32 of the corresponding x register.

The Zero Register: `xzr` and `wzr`

- Encoded as register number 31 within standard ALU instruction encodings.
- **Read Behavior:** Always returns constant zero (`0` for `xzr`, 32-bit `0` for `wzr`).
- **Write Behavior:** Values written to it are discarded.
- **Architectural Benefit:** Lets several common operations reuse existing instruction formats and supports convenient aliases such as `cmp`.

Common Idioms Using the Zero Register

Assembly Instruction	Equivalent Operation	Purpose
`mov x0, xzr`	$x_0 \leftarrow 0$	Clear register x_0 to zero
`subs xzr, x1, x2`	Evaluate $x_1 - x_2 \rightarrow$ flags	Compare x_1 and x_2 (alias: `cmp x1, x2`)
`str xzr, [x1]`	Mem$[x_1] \leftarrow 0$	Zero out a 64-bit memory word

Special-Purpose Registers: `sp`, `pc`, `lr`

Stack Pointer (`sp`)

- Dedicated register holding stack top address.
- Must maintain **16-byte alignment** in the ABI.
- Primarily tracks the current stack location; selected arithmetic and load/store instructions permit `sp` as an operand.

Program Counter (`pc`)

- Represents the address associated with current instruction execution.
- **Cannot** be read/written as data (no `mov x0, pc`).
- PC-relative addresses are formed with instructions such as `adr` and page-relative `adrp`.
- Modified via branch/call instructions.

Link Register (`x30` / `lr`)

`x30` is a general-purpose register conventionally used as the link register. `bl` and `blr` place the return address in `x30`; `ret` normally returns through it.

The Calling Convention: Caller- vs. Callee-Saved

Caller-Saved Registers (`x0`–`x17`)

- Used for arguments and temporary scratch math.
- Caller **must save them** to stack before calls if values are needed later.
- The ABI does not require the callee to preserve their incoming values.

Callee-Saved Registers (`x19`–`x28`)

- Preserves long-lived variables across calls.
- If callee modifies these, it **must save and restore** their original contents.
- Guarantees protection of caller state.

The Efficiency Trade-off

Balancing caller- and callee-saved registers across the ABI minimizes memory traffic and stack spilling during nested subroutine calls.

AArch64 Register File Overview

Register	Architectural Role	Preserved Across Calls? (ABI)
`x0–x7`	Function arguments & return values	No (Caller-saved / Scratch)
`x8`	Indirect result location (struct return pointer)	No (Caller-saved)
`x9–x15`	Temporary / Scratch registers	No (Caller-saved)
`x16–x17`	Intra-procedure-call temporary registers (IP0, IP1)	No (Caller-saved)
`x18`	Platform register (reserved for OS / TLS)	Platform-dependent
`x19–x28`	Callee-saved registers	**Yes (Callee-saved)**
`x29` (`fp`)	Frame pointer (base of current stack frame)	**Yes (Callee-saved)**
`x30` (`lr`)	Link register (subroutine return address)	No (Caller-saved)
`sp`	Stack pointer; ABI alignment must be maintained	ABI invariant
`xzr`	Dedicated zero register (constant 0, writes discarded)	N/A (Hardware constant)

Process State: `pstate` and Condition Flags

- The processor maintains architectural processor state in `pstate`, including the NZCV condition flags.
- The upper 4 bits evaluate the **NZCV Arithmetic Condition Flags**:

Flag	Name	Condition Under Which Flag Is Set to 1
N	**Negative**	Result bit 63 (or bit 31) is 1 (negative signed value)
Z	**Zero**	Computed result is exactly zero
C	**Carry**	Operation generated an unsigned carry-out (or no borrow in sub)
V	**Overflow**	Signed result cannot be represented in the destination width

Selective Flag Updates

Standard ALU instructions (add, sub) do **not** modify flags. Only instructions with an "s" suffix (`adds`, `subs`) or comparison instructions (`cmp`) update NZCV.

PART 3

Memory Model, Alignment & Endianness

Memory Organization & Byte Addressing

- Memory is byte-addressed and exposed to software as a sequence of **8-bit bytes**.
- Addresses are represented in 64-bit registers, although implemented virtual-address ranges are smaller than the full 64-bit space.
- Data operands span consecutive byte addresses scaling by standard powers of two:

Data Type	Size (Bytes)	Size (Bits)	AArch64 Architecture Term
Byte	1	8-bit	Byte (b)
Halfword	2	16-bit	Halfword (h)
Word	4	32-bit	Word (w)
Doubleword	8	64-bit	Doubleword (x)
Quadword	16	128-bit	Quadword / Vector element (q)

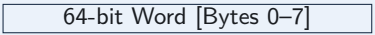
64-Bit Pointer Translation

While pointers are 64 bits wide, implemented virtual and physical address sizes are smaller and configuration-dependent; common systems use ranges such as 48 or 52 bits.

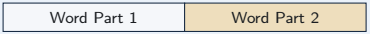
Memory Alignment Rules

- An access to an n-byte operand is **naturally aligned** if Address $(\mathrm{mod}\ n) = 0$.
- **Alignment Rules:** Word (4B boundary), Doubleword (8B boundary), Instruction fetch (4B boundary).

Aligned Access (Single Transaction)



Unaligned Access (Crosses Boundary)



Alignment Penalties

Many ordinary accesses to Normal memory can be unaligned, although an unaligned access may require additional memory transactions or cycles. Some access types, memory attributes, or configured environments require alignment and can generate an alignment fault.

Byte Ordering: Little-Endian vs. Big-Endian

Storage of 32-bit Word `0x0a0b0c0d` at Base Address `0x1000`:

Little-Endian (AArch64 Default)

- **Least Significant Byte (LSB)** stored at the **lowest** memory address.

Address	Stored Byte
`0x1000`	`0x0d` (LSB)
`0x1001`	`0x0c`
`0x1002`	`0x0b`
`0x1003`	`0x0a` (MSB)

Big-Endian (Network Order)

- **Most Significant Byte (MSB)** stored at the **lowest** memory address.

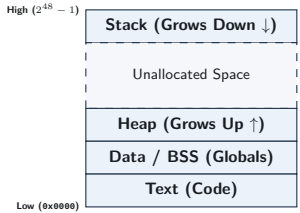
Address	Stored Byte
`0x1000`	`0x0a` (MSB)
`0x1001`	`0x0b`
`0x1002`	`0x0c`
`0x1003`	`0x0d` (LSB)

Network Interoperability

Multi-byte network protocol fields conventionally use network byte order (big-endian). Software can convert them with instructions such as `rev`, library routines, or explicit byte-wise parsing.

The Process Address Space Layout

- **Text (Code):** Read-only machine instructions.
- **Data / BSS:** Globally accessible and static variables.
- **Heap:** Dynamically allocated memory via `malloc`; grows **upward** toward higher addresses.
- **Stack:** Local function variables and saved caller state; conventionally grows **downward** toward lower addresses.



Simplified Address-Space Model

Modern processes also contain shared libraries, memory-mapped files, anonymous mappings, guard pages, and ASLR-created gaps. Excessive recursion typically reaches a stack limit or guard page and faults.

PART 4

Foundational Instruction Classes

Instruction Syntax Conventions

- Many AArch64 data-processing instructions use the form: `mnemonic dst, src1, src2_or_imm`
- **Three-Operand Format:** Computation preserves source registers without overwriting them.

Assembly Example	RTL Operation	Description
`add x0, x1, x2`	$x_0 \leftarrow x_1 + x_2$	64-bit register addition
`add w0, w1, w2`	$w_0 \leftarrow w_1 + w_2$	32-bit addition (clears upper 32 bits of x_0)
`add x0, x1, #16`	$x_0 \leftarrow x_1 + 16$	Addition with immediate constant

Explicit Destination Operand

Many AArch64 data-processing instructions use a separate destination register, so source registers need not be overwritten. This differs from traditional two-operand scalar x86 forms, while newer x86 vector extensions also support three-operand forms.

Arithmetic Instructions: Addition & Subtraction

Instruction	Operation	Flags Updated?
`add xd, xn, xm`	$x_d \leftarrow x_n + x_m$	No
`adds xd, xn, xm`	$x_d \leftarrow x_n + x_m$	**Yes (Updates NZCV)**
`sub xd, xn, xm`	$x_d \leftarrow x_n - x_m$	No
`subs xd, xn, xm`	$x_d \leftarrow x_n - x_m$	**Yes (Updates NZCV)**
`neg xd, xm`	$x_d \leftarrow 0 - x_m$ (Alias: `sub xd, xzr, xm`)	No
`negs xd, xm`	$x_d \leftarrow 0 - x_m$ (Alias: `subs xd, xzr, xm`)	**Yes**

Immediate Scaling

Immediate values (`#imm12`) span 0 to 4095 and can optionally be encoded with a 12-bit left shift, allowing selected larger constants to be represented in one instruction.

Logical Operations: Bitwise Manipulation

- Bitwise operations execute concurrently across all 64 parallel bit slices in the ALU.

Mnemonic	Operation	Bitwise Logic
`and xd, xn, xm`	Bitwise AND	$x_d \leftarrow x_n \wedge x_m$
`orr xd, xn, xm`	Bitwise OR	$x_d \leftarrow x_n \vee x_m$
`eor xd, xn, xm`	Bitwise XOR	$x_d \leftarrow x_n \oplus x_m$
`bic xd, xn, xm`	Bit Clear (AND NOT)	$x_d \leftarrow x_n \wedge (\sim x_m)$
`mvn xd, xm`	Bitwise NOT	$x_d \leftarrow \sim x_m$
`ands xd, xn, xm`	Bitwise AND with Flags	$x_d \leftarrow x_n \wedge x_m \rightarrow$ NZCV

Bit Clear (`bic`) Utility

`bic x0, x1, x2` acts as a hardware mask, clearing every bit in x_1 where the corresponding bit in x_2 is 1.

Shift Operations: Barrel Shifter Integration

- AArch64 hardware integrates a dedicated **barrel shifter** onto the second operand datapath.
- Shift instructions execute standalone or fold directly into primary ALU operations.

Mnemonic	Operation Name	Mathematical Meaning
`lsl xd, xn, #k`	Logical Shift Left	$x_d \leftarrow x_n \ll k$ (Multiplication by 2^k)
`lsr xd, xn, #k`	Logical Shift Right	$x_d \leftarrow x_n \gg k$ (Unsigned division by 2^k, zero-fill)
`asr xd, xn, #k`	Arithmetic Shift Right	$x_d \leftarrow x_n \gg k$ (Signed division by 2^k, sign-fill)
`ror xd, xn, #k`	Rotate Right	Circular right bit rotation

Integrated Shifted Register Example

`add x0, x1, x2, lsl #3` $\implies x_0 \leftarrow x_1 + (x_2 \times 8)$ encodes the shifted register operand as part of a single AArch64 instruction.

Comparison and Testing Instructions

- Comparison instructions evaluate operands, update NZCV flags, and discard the numerical result.

Instruction	Underlying Operation	Flags Updated
`cmp xn, xm`	`subs xzr, xn, xm`	Evaluates $x_n - x_m \rightarrow$ NZCV
`cmn xn, xm`	`adds xzr, xn, xm`	Evaluates $x_n + x_m \rightarrow$ NZCV (Compare Negative)
`tst xn, xm`	`ands xzr, xn, xm`	Evaluates $x_n \wedge x_m \rightarrow$ NZCV (Bit Test)

Flag Interpretation Rules

If $x_n == x_m$, `cmp` asserts **Z (Zero)** since $x_n - x_m = 0$. If $x_n < x_m$ (signed), `cmp` configures flags such that N $\neq$ V (Negative differs from Overflow).

Loading Immediate Constants: `movz`, `movk`, `movn`

- A fixed 32-bit instruction cannot fit an arbitrary 64-bit constant.
- AArch64 constructs large constants in 16-bit chunks using **wide-immediate instructions**:

Instruction	Operation Name	Semantic Action
`movz xd, imm, lsl s`	Move Wide with Zero	Places 16-bit immediate at shift s; others zero
`movk xd, imm, lsl s`	Move Wide with Keep	Replaces 16 bits at shift s; preserves others
`movn xd, imm, lsl s`	Move Wide with NOT	NOT of shifted 16-bit immediate forms destination

Constructing a 32-bit Value (`0x1234abcd`)

```
movz x0, #0xabcd            x0 = 0x000000000000abcd
movk x0, #0x1234, lsl #16   x0 = 0x000000001234abcd
```

Memory Access: Basic Load/Store Architecture

- Ordinary integer memory access uses explicit load and store instructions such as `ldr` and `str`.
- The instruction form and destination/source register width determine the transfer size:

Instruction	Register	Transfer Width	Action
`ldr xt, [xn]`	64-bit (`xt`)	8 Bytes (Doubleword)	$x_t \leftarrow \text{Mem}_8[x_n]$
`str xt, [xn]`	64-bit (`xt`)	8 Bytes (Doubleword)	$\text{Mem}_8[x_n] \leftarrow x_t$
`ldr wt, [xn]`	32-bit (`wt`)	4 Bytes (Word)	$w_t \leftarrow \text{Mem}_4[x_n]$ (Zero-extended)
`str wt, [xn]`	32-bit (`wt`)	4 Bytes (Word)	$\text{Mem}_4[x_n] \leftarrow w_t$
`ldrb wt, [xn]`	8-bit (`wt`)	1 Byte	$w_t \leftarrow \text{Mem}_1[x_n]$ (Zero-extended)
`strb wt, [xn]`	8-bit (`wt`)	1 Byte	$\text{Mem}_1[x_n] \leftarrow w_t[7:0]$

Pointer Referencing

The base register (x_n) holds the 64-bit memory pointer, explicitly enclosed in brackets `[xn]` to denote architectural indirection.

Addressing Modes: Base with Immediate Offset

Mechanism (EA = x_n + offset)

- Syntax: `ldr xt, [xn, #offset]`
- Accesses memory at x_n + offset.
- **No side effects:** Base register x_n remains unchanged post-execution.

Code Examples

`ldr x1, [x0]`	Load $[x_0]$
`ldr x2, [x0, #8]`	Load $[x_0 + 8]$
`str x2, [x0, #16]`	Store $[x_0 + 16]$

Hardware Offset Range

In the scaled unsigned-immediate form, a 12-bit offset field is multiplied by the access size. For 64-bit doublewords, the scale factor is 8, giving a maximum offset of $+32760$ bytes. Other load/store forms also support signed unscaled offsets.

Advanced Addressing: Pre- and Post-Indexed

Pre-Indexed (!)

```
ldr xt, [xn, #imm]!
```

1. Update base: $x_n \leftarrow x_n + \text{imm}$.
2. Load memory: $x_t \leftarrow \text{Mem}[x_n]$.

Base update happens *before* access.

Post-Indexed

```
ldr xt, [xn], #imm
```

1. Load memory: $x_t \leftarrow \text{Mem}[x_n]$.
2. Update base: $x_n \leftarrow x_n + \text{imm}$.

Base update happens *after* access.

Efficiency Impact

Pointer updates can be integrated directly into memory instructions, which can avoid a separate pointer-update instruction when traversing arrays or managing stack frames.

Loading and Storing Pairs (`ldp` and `stp`)

- **Core Purpose:** Transfer two adjacent 64-bit registers in a single instruction.
- **Primary Use Case (Stack Management):**
  - Encodes two adjacent register transfers in one architectural instruction.
  - Saves/restores Frame Pointer (x_{29}) and Link Register (x_{30}) efficiently during calls.
- **Implementation Note:** The underlying memory transactions and timing are implementation-dependent; `ldp`/`stp` do not imply general 128-bit atomicity.

Stack Frame Construction Example

```
// Save FP and LR; decrement sp by 16 bytes
stp x29, x30, [sp, #-16]!
```

Control Flow: Unconditional Branches

- Control flow instructions redirect program execution by modifying the Program Counter (pc).

Instruction	Name / Syntax	Operational Behavior
`b label`	Direct Branch	pc $\leftarrow$ label (PC-relative range: ± 128 MB)
`br xn`	Indirect Branch	pc $\leftarrow x_n$ (Jump to address held in register x_n)
`bl label`	Branch with Link	$x_{30} \leftarrow$ pc $+ 4$; pc $\leftarrow$ label (Subroutine call)
`blr xn`	Indirect Call	$x_{30} \leftarrow$ pc $+ 4$; pc $\leftarrow x_n$ (Function pointer call)
`ret`	Subroutine Return	pc $\leftarrow x_{30}$ (Default return via Link Register)

PC-Relative Addressing

Direct branch offsets are encoded as signed displacements relative to the current pc. This supports position-independent code within the instructionś encoded displacement range.

Conditional Branches & Condition Codes

Syntax: `b.cond label` (Branches to `label` only if the condition evaluates to true).

Condition Code	Meaning	Flag Condition	Comparison Context
`b.eq`	Equal / Zero	$Z == 1$	Signed / Unsigned
`b.ne`	Not Equal / Non-Zero	$Z == 0$	Signed / Unsigned
`b.lt`	Signed Less Than	$N \neq V$	Signed
`b.le`	Signed Less Than or Equal	$(Z == 1) \vee (N \neq V)$	Signed
`b.gt`	Signed Greater Than	$(Z == 0) \wedge (N == V)$	Signed
`b.ge`	Signed Greater or Equal	$N == V$	Signed
`b.lo`	Unsigned Lower ($<$)	$C == 0$	Unsigned
`b.hi`	Unsigned Higher ($>$)	$(C == 1) \wedge (Z == 0)$	Unsigned

Branch Prediction Context

A mispredicted conditional branch can disrupt speculative execution and require younger instructions to be discarded and refetched. Processors reduce this cost with hardware branch predictors and branch target buffers (BTBs).

Control Flow Idiom: If-Else Translation

High-Level C Code

```
if (a == b) {
    c = c + 1;
} else {
    c = c - 1;
}
```

AArch64 Assembly Mapping

```
// Assume: x0=a, x1=b, x2=c
cmp x0, x1        // Compare a and b
b.ne .Lelse       // If a != b, jump to else
add x2, x2, #1    // Then-body: c = c + 1
b .Lend           // Skip else-body
.Lelse:
sub x2, x2, #1    // Else-body: c = c - 1
.Lend:
```

Assembly Inversion Pattern

Compilers often invert a condition so the branch skips over the preferred fall-through path and transfers control directly to the alternative path when needed.

Control Flow Idiom: While-Loop Translation

High-Level C Loop

```
// Compute sum of array
uint64_t i = 0;
while (i < n) {
    sum += arr[i];
    i++;
}
```

AArch64 Assembly Translation

```
// x0=arr, x1=n, x2=i, x3=sum
mov x2, xzr                 // i = 0
mov x3, xzr                 // sum = 0
.Lloop:
cmp x2, x1                  // Test i < n
b.hs .Ldone                 // Exit if i >= n (unsigned)
ldr x4, [x0, x2, lsl #3]    // Load arr[i]
add x3, x3, x4              // sum += arr[i]
add x2, x2, #1              // i++
b .Lloop                    // Repeat loop
.Ldone:
```

Optimization Note

Compilers may unroll loop bodies to reduce branch frequency and amortize loop-control work across multiple iterations.

Conditional Select Instructions: Avoiding Selected Branches

- Conditional branches can incur a recovery cost when prediction is incorrect.
- AArch64 provides **conditional select instructions** that can replace a branch for suitable short conditional expressions:

Instruction	Semantic Operation	C Ternary Equivalent
`csel xd, xn, xm, cond`	$x_d \leftarrow \text{cond} \,?\, x_n : x_m$	`d = cond ? n : m;`
`cinc xd, xn, cond`	$x_d \leftarrow \text{cond} \,?\, (x_n + 1) : x_n$	`d = n + (cond ? 1 : 0);`
`cset xd, cond`	$x_d \leftarrow \text{cond} \,?\, 1 : 0$	`d = (cond) ? 1 : 0;`

Branchless Maximum: $\max(x_1, x_2)$

```
cmp x1, x2              // Compare x1 and x2
csel x0, x1, x2, gt     // If x1 > x2, x0 = x1; else x0 = x2
```

Review and Self-Test Questions

Topic 3: Comprehensive Summary

- **Architectural State:** Precise hardware-software contract defined by general registers (`x0`–`x30`), flags (NZCV), stack pointer (`sp`), program counter (`pc`), and byte-addressed memory.
- **Register Structure:** 31 highly regular 64-bit general-purpose registers (`x0`–`x30`) have corresponding 32-bit views (`w0`–`w30`); writing a `w` register clears the upper 32 bits of the corresponding `x` register.
- **ABI Rules:** `x0`–`x7` handle arguments/returns (caller-saved); `x19`–`x28` hold local variables across calls (callee-saved).
- **Load/Store Operations:** Integer ALU operations use registers, while memory is accessed with explicit load/store instruction classes, including offset, indexed, and paired forms.
- **Control Flow:** Branching can evaluate NZCV flags updated by `adds`/`subs`/`cmp`; conditional select (`csel`) can avoid a conditional branch for suitable expressions.

Self-Test Questions: State & Register Mechanics

1. **Architectural vs. Microarchitectural State:** Why are CPU branch prediction tables and physical register rename structures classified as microarchitectural state rather than architectural state?
2. **32-bit Register Write Semantics:** Suppose register `x5` initially contains `0xffffffff_ffffffff`. If the instruction `mov w5, #0x1234` is executed, what exact 64-bit value resides in `x5` post-execution?
3. **Zero Register Application:** Why does AArch64 provide a dedicated zero register (`xzr`/`wzr`) instead of permanently assigning a general-purpose register (like `x0`) to hold zero?
4. **Calling Convention Responsibility:** If function `foo()` calls `bar()`, and `foo()` needs to preserve values across the call in registers `x3` and `x20`, which function is responsible for saving each register to the stack?

Self-Test Questions: Memory & Instruction Mechanics

1. **Memory Addressing Modes:** Explain the mechanical difference in address calculation and base register updates between: `ldr x1, [x0, #16]` vs. `ldr x1, [x0, #16]!` vs. `ldr x1, [x0], #16`.
2. **Endianness Evaluation:** The 32-bit word `0xdeadbeef` is stored at address `0x2000` on a little-endian AArch64 machine. What exact byte is stored at address `0x2000`? At address `0x2003`?
3. **Branch Elimination via `csel`:** Convert this C statement into branchless assembly using `cmp` and `csel`:
   ```
   int64_t result = (val1 <= val2) ?  (val1 + 10) :  (val2 - 5);
   ```
4. **Condition Flag Verification:** If `x0` contains 5 and `x1` contains 10, which NZCV flags are asserted $(= 1)$ following `cmp x0, x1`?

Looking Ahead to Topic 4: Assembly Programming

Topic 4 Preview (Arithmetic, Logic, & Control Flow)

Moving from architectural structure to assembly implementation.

- **ALU & Shifted Operands:** Arithmetic using shifted-register operands integrated into AArch64 data-processing instructions.
- **Bitwise Logic & Field Extraction:** Manipulating bitfields using masks, bit clears (`bic`), and bitfield extracts (`ubfx`).
- **Condition Flags & Comparisons:** Leveraging NZCV flags and choosing between signed vs. unsigned branch conditions.
- **Control Flow & Conditional Selection:** Translating loops and nested conditionals while using `csel` to reduce reliance on conditional branches for suitable expressions.